Examen CS S1 08_01_2020

January 16, 2020

1 Exercice 1 : (7 points)

Question 1 (0.5 point) : Importer la bibliothèque python destinée à manipuler des matrices ou des tableaux multidimensionnels ainsi que des fonctions mathématiques opérant sur ces derniers. Donner l'instruction nécessaire.

```
In [1]: import numpy as np
        #from numpy import *
```

Question 2 (1 point) : On considère une fonction f tabulée, définie par la connaissance des points $(x_i, y_i = f(x_i))_{0\leq i\leq 8}$ illustrés sur le tableau ci-dessous :

i	0	1	2	3	4	5	6	7	8
x_i	-2	$\frac{-3}{2}$	-1	$\frac{-1}{2}$	0	$\frac{1}{2}$	1	$\frac{3}{2}$	2
$y_i = f(x_i)$	$\frac{1}{17}$	$\frac{1}{10}$	$\frac{1}{5}$	$\frac{1}{2}$	1	$\frac{1}{2}$	$\frac{1}{5}$	$\frac{1}{10}$	$\frac{1}{17}$

Créer deux listes X et Y composées, respectivement, des $(x_i)_{0\leq i\leq 8}$ et $(y_i)_{0\leq i\leq 8}$. Donner les instructions nécessaires.

```
In [2]: X=np.array([-2,-3/2,-1,-1/2,0,1/2,1,3/2,2])
        Y=np.array([1/17,1/10,1/5,1/2,1,1/2,1/5,1/10,1/17])
```

Question 3 (1.5 point) : On souhaite interpoler les points $(x_i, y_i)_{0\leq i\leq 8}$ de la question 2 par la méthode de Lagrange. Pour ce faire, commencez par créer une fonction `Interpolation(t,X,Y)` évaluant le polynôme d'interpolation P, exprimé dans la base de polynômes de Lagrange et associé aux points d'abscisses $X = (x_i)_{0\leq i\leq n}$ et d'ordonnées $Y = (y_i)_{0\leq i\leq n}$, en une liste de points `t`. L'expression de P est rappelée ci-dessous :

$$P(t) = \sum_{i=0}^{n} y_i L_i(t), \quad t \in \mathbb{R}, \quad n \in \mathbb{N}^*,$$

où $L_i(t) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{t - x_j}{x_i - x_j}$ désigne le polynôme de Lagrange associé au point x_i, $0 \leq i \leq n$.

Utiliser une flèche $\longrightarrow$ pour indiquer une indentation (tabulation).

```
In [3]: def Lagrange(t,i,x):
            n=len(x)
            L=1
            for j in np.arange(0,n):
                if j!=i:
                    L*=(t-x[j])/(x[i]-x[j])
            return L

        def Interpolation_Lagrange(t,x,y):
            n=len(x)
            P=np.zeros((len(t)))

            for i in np.arange(0,n):
                P+=y[i]*Lagrange(t,i,x)
            return P
        ######################################################
        def interpolation_lagrange(t,x,y):
            n=len(x)
            P=np.zeros((len(t)))
            for i in np.arange(0,n):
                L=1
                for j in np.arange(0,n):
                    if j!=i:
                        L*=(t-x[j])/(x[i]-x[j])
                P+=y[i]*L
            return P
```

Question 4 (2 points) :

1. Considérer 100 points répartis uniformément sur $[-2, 2]$ et stokés dans une variable `t`. Donner l'instruction nécessaire.
2. Importer la bibliothèque python destinée à tracer et visualiser des données sous formes de graphiques. Donner l'instruction nécessaire.
3. Sur une même figure, représenter le polynôme interpolant les points de la question 2 et évalué en `t`, en bleu avec une ligne continue d'épaisseur égal à 2, et les points d'interpolation en utilisant des 'markers' en étoiles rouges de tailles égales à 12. Donner les instructions nécessaires sans reproduire les figures.

```
In [4]: t=np.linspace(-2,2,100)
        import matplotlib.pyplot as plt
        P=Interpolation_Lagrange(t,X,Y)
        plt.figure(figsize=(20,10))
        plt.plot(t,P,'b-',X,Y,'r*',linewidth=2,markersize=12)

Out[4]: [<matplotlib.lines.Line2D at 0x7f6d898>,
         <matplotlib.lines.Line2D at 0x7f6d9e8>]
```

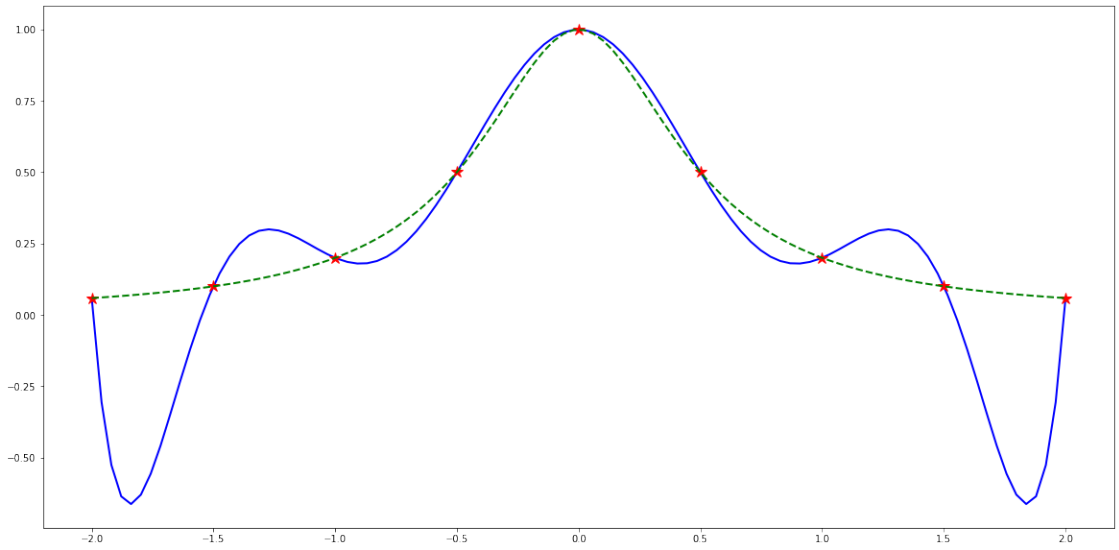
Question 5 (0.5 point) : Soit la fonction f définie sur $[-2,2]$ par $f(t) = \dfrac{1}{1+4t^2}$.
Ecrire la fonction f.

```
In [5]: f=lambda t: 1/(1+4*t**2)
```

Question 6 (1.5 points) : Sachant maintenant que les données représentées dans le tableau de la question 2, sont issues de la fonction f de la question 5, rajouter à la figure de la question 4, la représentation graphique de f évaluée en `t` avec une ligne discontinue en vert et d'épaisseur 2. Donner les instructions nécessaires sans reproduire les figures. Observer ensuite intérpréter le résultat en terme de qualité de l'interpolation. Donner une explication du résultat obtenu.

```
In [6]: plt.figure(figsize=(20,10))
        plt.plot(t,P,'b-',X,Y,'r*',t,f(t),'g--',linewidth=2,markersize=12)

Out[6]: [<matplotlib.lines.Line2D at 0x87c7240>,
         <matplotlib.lines.Line2D at 0x87c7390>,
         <matplotlib.lines.Line2D at 0x87c7710>]
```



2 Exercice 2 : (6 points)

Question 1 (2 points) : On considère la fonction f définie sur $[1,2]$ par :

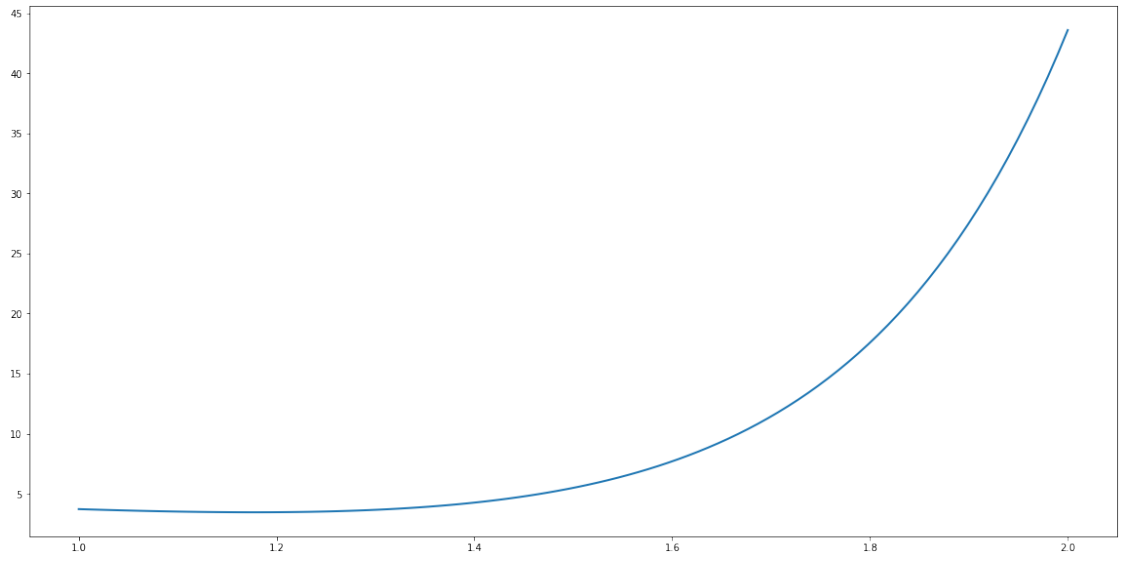
$$f(x) = e^{x^2} - 4x^2 + 5.$$

En se basant sur la représentation graphique de f sur l'intervalle $[1,2]$, et en **justifant votre réponse** :

1. Déterminer le nombre de racines de l'équation $f(x) = 0$.
2. Donner le signe de f', la dérivée première de f.

```
In [7]: g=lambda x: np.exp(x**2)-4*x**2+5
        x=np.linspace(1,2,100)
        plt.figure(figsize=(20,10))
        plt.plot(x,g(x),linewidth=2,markersize=12)

Out[7]: [<matplotlib.lines.Line2D at 0x8c4a860>]
```



L'equation n'admet pas de racines car d'après la courbe de f, la fonction f est strictement positive.\ La fonction f est croissante, par conséquent, $f^{'}$ est positive.

Question 2 (2 points) : En utilisant la fonction `append()`, créer une fonction nommée `puissance(n)`, qui, pour un entier nautrel $n \in \mathbb{N}^*$, retourne une liste de taille n dont l'élement d'indice i contient l'entier 2^i pour i allant de 0 à $n-1$.

Utiliser une flèche $\longrightarrow$ pour indiquer une indentation (tabulation).

```
In [8]: def puissance(n):
            L=[]
            for i in range(n):
                L.append(2**i)
            return L
```

Question 3 (2 points) : Corriger le code suivant pour qu'il affiche la matrice $A = (a_{i,j})_{1 \leq i,j \leq n}$ d'ordre $n \in \mathbb{N}^*$ telle que :

$$a_{i,j} = i + j, \quad \forall 1 \leq i, j \leq n.$$

Utiliser une flèche $\longrightarrow$ pour indiquer une indentation (tabulation).

```
In [9]: def matrice(n):
            A=np.ones(n)
            for i in np.arange(n):
                for j in np.arange(n):
                A[i,j]=i+j+2
                return A


          File "<ipython-input-9-f32754e2fd25>", line 5
        A[i,j]=i+j+2
        ^
    IndentationError: expected an indented block
```

```
In [ ]: def matrice(n):
            A=U=np.ones((n,n))
            for i in np.arange(n):
                for j in np.arange(n):
                    A[i,j]=i+j+2
            return A
```

```
In [ ]: matrice(3)
```

3 Exercice 3 : (7 points)

Sous certaines conditions, la concentration du glucose G dans le sang, exprimée en gramme par litre (g/l), satisfait l'Equation Différentille Ordinaire (EDO) $^{(*)}$ suivante :

$$G'(t) = r - KG(t), \;\; t \geq 0, \;\; r > 0, et \;\; K > 0, \quad (1)$$

où t désigne le temps, exprimé en **minutes**, r correspond au taux d'infusion du glucose dans le sang, exprimé en gramme par minute (g/mn), et K est une constante de proportionnalité.

L'objectif de cet exercice est d'observer, durant une période de 3 heures, l'évolution de la concentration du glucose dans le sang d'un patient admettant les paramètres suivants :

$r = 0.1$ g/mn,
$K = 0.05$,
$G(0) = 3$ g/l.

$\forall t \in [0, T]$, l'EDO (1) devient alors :

$$G'(t) = 0.1 - 0.05G(t), \quad G(0) = 3, \quad (2)$$

où T désigne la période d'observation de G exprimée en minutes.

Pour observer l'évolution de G de ce patient, nous passons par une résolution numérique du problème de Cauchy (2) par la méthode de Runge Kutta d'ordre 4 (RK4) dont le code est rappelé dans l'annexe.

$^{(*)}$ L'EDO (1) est donnée en négligeant certaines contraintes liées à la physiologie du patient et à sa consommation du glocose.

Question 1 (1.5 point) : En écrivant l'EDO (2) sous la forme classique d'un problème de Cauchy :

$$G' = f(t, G), \quad G(t_0) = G_0, \quad t \in [t_0, t_0 + T],$$

identifier les paramaètres t_0, G_0, T, et la fonction $f(t, G)$ relatives à l'EDO (2).

t0=0, G0=3, T=180

Question 2 (1 point) : Ecrire la fonction $f(t, G)$ relative à l'EDO (2).

```
In [ ]: f=lambda t, G: 0.1-0.05*G
```

Question 3 (1 point) : Discrétiser l'intervalle $[t_0, t_0 + T]$ uniformément avec un pas $h = 3$ (3 mn). Stoker les valeurs obtenues en une variable nommée `t`. Donner l'instruction nécessaire et donner le nombre **entier** N de sous-intervalles de $[t_0, t_0 + T]$.

```
In [ ]: tt=np.linspace(0,180,61)
        tt=np.arange(0,181,3)
        n=60
```

```
In [ ]: tt
```

Question 4 (2 points) : Appliquer la méthode RK4 (voir annexe) pour approcher la concentration du gluose dans le sang de ce patient aux instants : $t = 60$ mn, $t = 120$ mn, e $t = 180mn$. Noter `G_app` la sortie de la fonction RK4. Donner les instructions nécessaires et les approximations affichées.

```
In [ ]: #G_app=RK4(lambda tt, G: 0.1-0.05*G, 3, 0,180,60)
        G_app=RK4(f, 3, 0,180,60)
        G_app[20], G_app[40], G_app[60]
```

Question 5 (1 point) : Nous donnons maintenant la solution exacte du problème de Cauchy (2) :

$$G(t) = 2 + e^{-0.05t}, \quad t \in [0, T]$$

Ecrire la fonction G.

```
In [ ]: G=lambda t: 2+np.exp(-0.05*t)
```

Question 6 (0.5 point) : Représenter graphiquement l'évolution de la concentration du glucose dans le sang du patient en étude. Sur un même graphique, tracer la courbe de G et celle des valeurs estimées (`G_app`). Sachant qu'une valeur de glucose supérieure à 1.5 g/l est alarmante, observer les graphiques (sans les reproduire) et dire si le patient en question a tendance à être diabétique. Justifier votre réponse.

```
In [ ]:
```

4 Annexe

Soit le problème de Cauchy suivant:

$$x' = f(t, x), \quad x(t_0) = x_0, \quad t \in [t_0, t_0 + T]$$

La fonction `RK4(f,x0,t0,T,N)` prend en paramètres la fonction f, la valeur initiale x_0, la valeur initiale de la subdivision t_0, la largeur de l'intervalle $[t_0, t_0 + T]$, T, sur lequel x est définie, et N le nombre de sous-intervalles de $[t_0, t_0 + T]$. Cette fonction retourner la liste des valeurs $[x_0, x_1, ..., x_N]$ calculées par le schéma de Runge Kutta d'ordre 4.

```
In [ ]: def RK4(f, x0, t0,T,N):
            x = [x0]
            h=T/N
            for k in np.arange(0, N):
                p1 = f(t0+k*h,x[k])
                p2 = f( t0+k*h + h / 2, x[k] + h * p1 / 2)
```

```
            p3 = f( t0+k*h + h / 2, x[k] + h * p2 / 2)
            p4 = f(t0+(k+1)*h,x[k] + h * p3)
            x.append(x[k] + h * (p1+2*p2+2*p3+p4) / 6)
        return x
```

In [ ]: